

RELATÓRIO DAS ATIVIDADES — DOCKER E DOCKER COMPOSE

Ana Carolina da Silveira

6.3 — ATIVIDADES PARCIAIS DO CENÁRIO 1 (VUE + NODE + MYSQL + REDIS + RABBITMQ)

1. Por que a api usa mysql, redis e rabbitmq por nome de serviço, e não por localhost

Dentro de um mesmo Compose, todos os serviços declarados entram automaticamente na mesma rede (aqui, `app_net`), e o Docker cria um DNS interno que resolve o nome de cada serviço para o IP do seu container. `localhost`, dentro de um container, aponta para o próprio container — não para o host nem para os demais serviços. Por isso a api deve se conectar a `mysql:3306`, `redis:6379` e `rabbitmq:5672` usando o nome do serviço como host: é esse nome que o DNS interno do Compose resolve. Se a api usasse `localhost`, ela tentaria falar consigo mesma e a conexão falharia.

Diagrama de rede (resumo): `[api] --DNS interno app_net--> [mysql] / [redis] / [rabbitmq]`. Todos dentro da mesma rede lógica; o navegador do host acessa apenas as portas publicadas (5173 e 3000).

2. Redis x RabbitMQ: quando usar cache e quando usar fila

Redis é um armazenamento em memória de chave-valor, usado para cache (acelerar leituras repetidas, ex.: resultado de uma consulta cara), sessões de usuário e filas leves/simples. Seu foco é velocidade de acesso a dados já calculados ou de curta duração.

RabbitMQ é um broker de mensagens (AMQP), usado para desacoplar produtores e consumidores de eventos: a api publica uma mensagem (ex.: "pedido criado") e o worker a consome de forma assíncrona, com garantias de entrega, reprocessamento e ordenação de fila. Seu foco é comunicação confiável entre serviços, não velocidade de leitura de dados.

Exemplo real: ao cadastrar um pedido, a api grava no MySQL, guarda o resultado consultado com frequência (ex.: catálogo de produtos) no Redis para não repetir a consulta ao banco, e

publica um evento "pedido criado" no RabbitMQ para que o worker envie um e-mail de confirmação de forma assíncrona, sem atrasar a resposta ao usuário.

7.4 — ATIVIDADES PARCIAIS DO CENÁRIO 2 (REACT + POSTGRES + NODE/EXPRESS/INERTIA + REDIS + RABBITMQ)

1. Diferença entre ports e expose

Ports publica a porta do container no host, tornando-a acessível de fora do Docker (ex.: pelo navegador na máquina do desenvolvedor). expose apenas declara que a porta é usada, deixando-a acessível somente para outros containers na mesma rede do Compose, sem publicá-la no host.

Exemplo do proxy: o serviço proxy (Nginx) usa ports: "8080:80", pois é a porta de entrada do sistema, acessada pelo navegador em `http://localhost:8080`. Já frontend e api usam expose: "5173" e expose: "3000" — elas só precisam ser alcançadas pelo proxy dentro da rede interna `app_net`, então não publicam porta no host.

8.2 — QUESTÕES NORTEADORAS DO CENÁRIO 3 (APLICADAS AO PROJETO ENTREGUE)

Quais serviços precisam ser acessados pelo host e quais ficam só na rede interna?

Frontend (5173) e API (3000) são acessados pelo host/navegador; RabbitMQ publica o painel de management (15672) para diagnóstico. PostgreSQL e Redis permanecem apenas na rede interna `app_net`, acessados só pela api e pelo worker.

Quais dados precisam sobreviver ao `docker compose down`? Os dados do PostgreSQL (tabela de eventos e demais dados de negócio) e, se usado para filas persistentes, o estado do Redis — por isso ambos usam volumes nomeados (`postgres_data`, `redis_data`), que só são removidos com `docker compose down -v`.

Quais variáveis podem ser públicas no `.env.example` e quais seriam segredos?

Podem ser públicas as variáveis que só indicam nomes/estrutura (`POSTGRES_DB`, nomes de usuário de exemplo, URLs com host de serviço). Senhas reais (`POSTGRES_PASSWORD`, `RABBITMQ_PASSWORD`) nunca devem ir para o repositório — no `.env.example` elas aparecem apenas como placeholder (ex.: `troque_esta_senha`).

Como provar que backend conversa com banco, Redis e mensageria? Pelo endpoint `/health` da api (que testa a conexão com Postgres e Redis) e pelos logs do worker mostrando o consumo de mensagens publicadas pela api na fila eventos do RabbitMQ (`docker compose logs -f worker`).

Como o ambiente roda em Windows 11 sem WSL, Linux e macOS? O README.md do projeto entregue detalha os três caminhos: Windows 11 usando Docker Desktop com backend Hyper-V (sem WSL2), Linux usando Docker Engine + Compose plugin, e macOS usando Docker Desktop com atenção à arquitetura arm64/amd64.

10.1 — ATIVIDADES TEÓRICAS

1. Diferença entre imagem e container (analogia + exemplo com Node.js)

Analogia: a imagem é como a planta de uma casa (ou uma classe, em programação orientada a objetos) — um modelo estático e imutável. O container é a casa construída a partir dessa planta (a instância dessa classe) — algo em execução, que pode ser criado, parado e destruído sem alterar a planta original.

Exemplo: a imagem `node:24-alpine` contém o runtime Node.js e o sistema base. Ao rodar `docker run node:24-alpine node -v`, o Docker cria um container (um processo isolado) a partir dessa imagem; é possível criar vários containers diferentes (rodando comandos ou aplicações diferentes) a partir da mesma imagem, sem duplicá-la em disco.

2. Por que containers não substituem boas práticas de segurança, versionamento e observabilidade

Containers isolam a execução e padronizam dependências, mas não corrigem código vulnerável, não impedem o vazamento de segredos versionados por engano, não garantem testes automatizados e não geram, por si só, logs estruturados ou métricas. Uma imagem mal configurada (rodando como root, usando latest, sem healthcheck) reproduz os mesmos riscos de segurança e operação que existiriam sem containers — a tecnologia organiza o ambiente, mas as práticas de segurança, versionamento e observabilidade continuam sendo responsabilidade da equipe.

3. Volume nomeado x bind mount — quando usar cada um

Volume nomeado é uma área de armazenamento gerenciada pelo próprio Docker, ideal para dados que precisam persistir e não fazem sentido versionar no repositório, como os arquivos de um banco de dados (ex.: `mysql_data`, `postgres_data`).

Bind mount conecta uma pasta real do host diretamente ao container, sendo ideal durante o desenvolvimento, para refletir instantaneamente as alterações de código-fonte no container (hot reload), como o mapeamento `./backend:/app` usado nos cenários deste material.

4. Por que localhost dentro de um container não representa o host nem outro container

Cada container tem sua própria pilha de rede isolada. localhost dentro de um container aponta para a interface de loopback daquele próprio container, não para a máquina host nem para outros containers. Para acessar outro container é preciso usar o nome do serviço (resolvido pelo DNS interno do Compose); para acessar o host a partir de um container, seria necessário usar um endereço especial (como `host.docker.internal`, quando suportado).

5. Redis como cache x RabbitMQ como broker de mensagens

Redis armazena dados em memória para leitura rápida e repetida (cache, sessão), sendo consultado sob demanda pela aplicação. RabbitMQ transporta mensagens entre serviços de forma assíncrona e desacoplada, garantindo que um evento publicado seja entregue e processado por um consumidor, mesmo que produtor e consumidor não estejam ativos ao mesmo tempo. Cache resolve "acesso repetido a dados"; broker resolve "comunicação assíncrona e confiável entre serviços".

6. Por que latest pode ser perigoso em ambientes profissionais

A tag latest aponta para a versão mais recente publicada pelo mantenedor da imagem no momento do build, que muda ao longo do tempo. Isso quebra a reprodutibilidade: dois desenvolvedores (ou o ambiente de produção e o de desenvolvimento) podem acabar rodando versões diferentes do mesmo serviço sem perceber, causando bugs difíceis de rastrear e dificultando rollback. Fixar uma versão (ex.: `postgres:17-alpine`) garante que todo mundo — e o pipeline de CI/CD — execute exatamente a mesma imagem.

7. Como o Compose melhora o onboarding de novos desenvolvedores

Com um `docker-compose.yml`, `.env.example` e README bem documentados, um novo integrante da equipe consegue subir toda a arquitetura (frontend, backend, banco, cache, mensageria) com poucos comandos (`cp .env.example .env && docker compose up -d`), sem precisar instalar manualmente cada dependência no próprio sistema operacional nem alinhar

versões com o restante do time. Isso reduz drasticamente o tempo entre a entrada do desenvolvedor e sua primeira contribuição funcional.

8. Três riscos de versionar .env com credenciais reais

Vazamento de senhas e chaves de API caso o repositório seja público ou comprometido, permitindo acesso não autorizado a bancos e serviços externos.

Dificuldade de rotação de segredos: uma senha versionada no histórico do Git continua acessível mesmo depois de "removida" do arquivo atual, exigindo reescrita de histórico ou troca da credencial.

Confusão entre ambientes: credenciais de produção podem acabar sendo usadas por engano em desenvolvimento (ou vice-versa) quando estão hardcoded e compartilhadas no mesmo arquivo versionado.